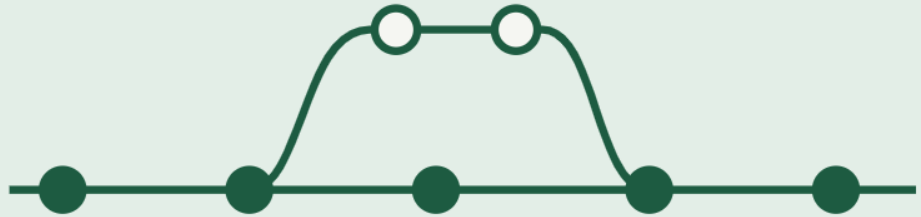
PEOPLE / IDEAS / PROGRESS

ALGOSIUM SOFTWARE

KNOWLEDGE RELAY · SHARE / LEARN / GROW

Git Workflow



From your first commit to running Git on large, resource-intensive projects

5th November 2026 · 3 to 4 PM

Anandu Babu

Lead Engineer – Software Systems

LEARN TODAY
BUILD TOMORROW

AGENDA

What we'll cover today

01

Git basics

Snapshots, the three areas, branches, merge and rebase

02

Branching strategies

Git Flow, GitHub Flow, trunk-based and how to choose

03

Commit conventions

Messages people can read and tools can parse

04

Git at scale

Large files, monorepos, fast clones and fast status

05

Problems & fixes

Real incidents and how to recover from them

06

Conflicts & tracking

Fewer conflicts, clean history, ticket-to-release traceability

How Git thinks

A commit is a full snapshot of your project, linked to the commit before it.

Distributed

Every clone is a complete repository with full history. You can commit, branch and diff offline.

Content-addressed

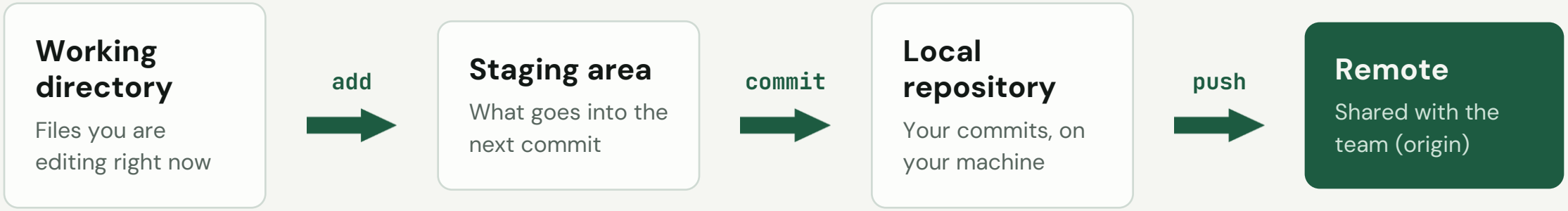
Each commit is named by a hash of its content and parents, so history can’t change silently.

Cheap branches

A branch is a tiny file pointing at one commit. Creating one costs nothing.

01 · GIT BASICS

Where your changes live



GOING THE OTHER WAY

`git restore <file>`
Discard edits in the working directory

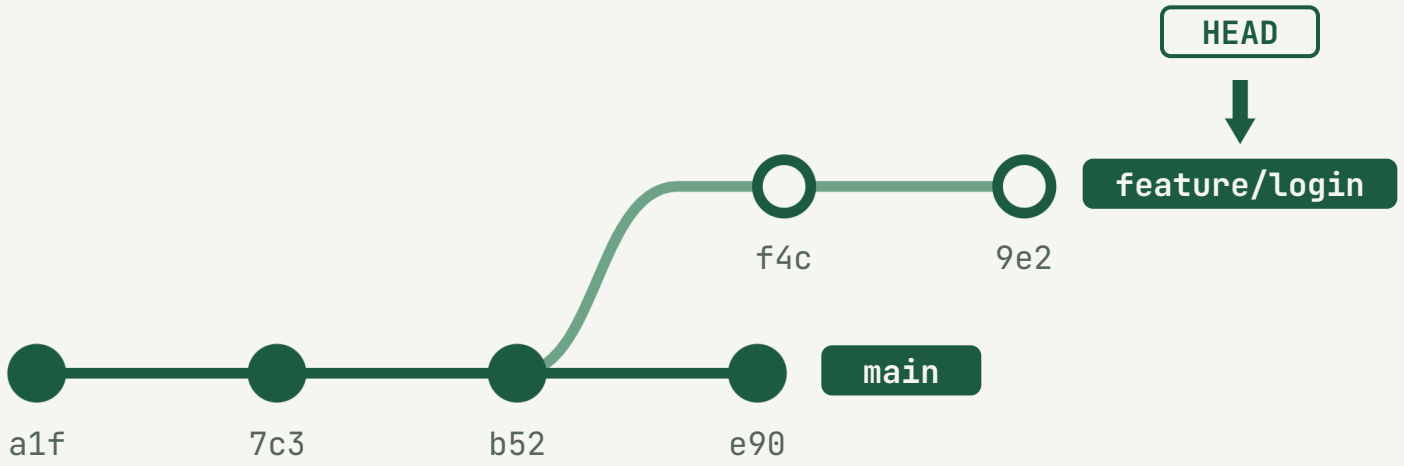
`git restore --staged`
Unstage a file but keep your edits

`git fetch / git pull`
Bring the team's commits down to you

The everyday commands

Start or copy a repository	`git init` · `git clone <url>`
See what changed	`git status` · `git diff`
Stage only the hunks you mean	`git add -p`
Record a snapshot	`git commit`
Read the history	`git log --oneline --graph --all`
Create and switch to a branch	`git switch -c feature/login`
Get the team’s latest work	`git fetch` · `git pull --rebase`
Share your branch	`git push -u origin feature/login`

A branch is just a pointer



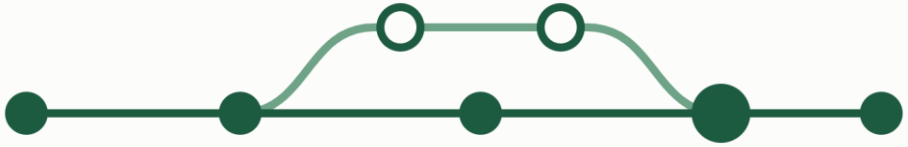
main and **feature/login** are labels that point at commits.

HEAD is where you are: the branch you have checked out.

Committing moves the current label forward. Nothing is copied.

Merge or rebase?

git merge



- Joins two histories with a merge commit
- Keeps the true record; never rewrites commits
- Use for shared branches and PR merges

git rebase



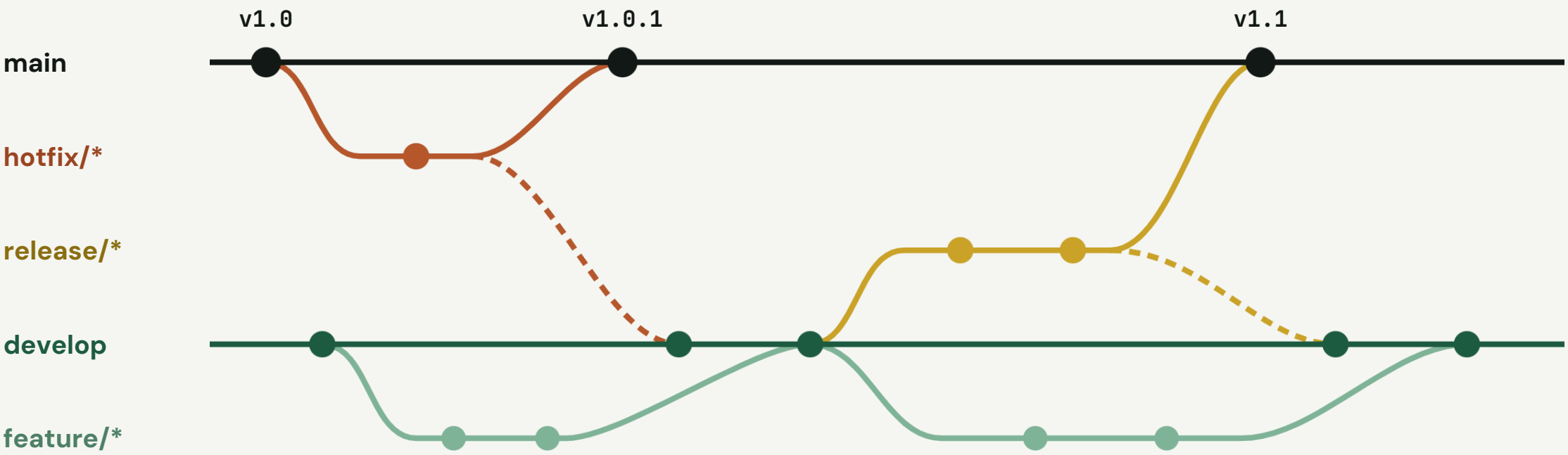
- Replays your commits on top of the latest main
- Linear history, but creates new commit hashes
- Use to update your own branch before a PR

Golden rule: rebase your own local work freely — never rebase commits others have already pulled.

Four common strategies

Strategy	How it works	Best for	Watch out for
Git Flow	main + develop, plus feature, release and hotfix branches	Versioned releases, several supported versions	Heavy process; long-lived branches drift
GitHub Flow	main is always deployable; short feature branches merged by pull request	Web apps and services with continuous deployment	Needs solid CI and fast reviews
Trunk-based	Everyone integrates into main at least daily; feature flags hide unfinished work	High-velocity teams and monorepos	Requires feature flags and strong tests
Release branches	Cut release/x.y from main; fixes land on main and are cherry-picked back	Products where customers run older versions	Back-porting discipline

Git Flow at a glance



Features land on **develop**. A **release** branch stabilises and merges into main and back. A **hotfix** patches main and flows back to develop.

Trunk-based development




```
if (flags.isOn("new-checkout")) {
renderNewCheckout();
}
```

Branches live hours

Small PRs, merged at least daily

Flags, not branches

Unfinished work ships dark behind a feature flag

main is releasable

CI runs on every push; a red main is fixed first

Release by tag

Tag main, or cut release/x.y only when needed

Choosing a strategy

IF

You deploy many times a day and run one version in production

USE

GitHub Flow or trunk-based

IF

You ship versioned releases and support older versions

USE

Git Flow or release branches

IF

Many teams share one large repo or monorepo

USE

Trunk-based + feature flags + CODEOWNERS

Whatever you choose: protect main, keep branches short-lived, and let CI decide what merges.

Name branches so they track

PREFIX	PURPOSE	EXAMPLE
`feature/`	New functionality	`feature/PAY-142-upi-refunds`
`fix/`	Bug fix in the normal flow	`fix/PAY-187-null-currency`
`hotfix/`	Urgent production patch	`hotfix/1.4.1-login-timeout`
`release/`	Stabilise a version	`release/1.5.0`
`chore/`	Tooling, dependencies, docs	`chore/bump-node-22`

Rules: lowercase and hyphens · include the ticket key · one purpose per branch · delete after merge

Conventional Commits

```
<type>(<scope>): <subject>

<body: what changed and why>

<footer: BREAKING CHANGE / Refs>
```

- Subject in the imperative: “add”, not “added”
- About 50 characters, no trailing period
- Body explains **why**; the diff already shows what
- Enforce it with commitlint in a commit-msg hook

feat	a new feature
fix	a bug fix
docs	documentation only
style	formatting, no logic change
refactor	restructure, same behaviour
perf	performance improvement
test	adding or fixing tests
build	build system, dependencies
ci	pipeline configuration
chore	other maintenance
revert	undoes an earlier commit

03 · COMMIT CONVENTIONS

Messages your team can use

AVOID	PREFER
fixed stuff	fix(auth): refresh token before expiry check
WIP	feat(payments): add UPI refund endpoint
update	perf(search): cache tokenizer per index
changes after review	refactor(api): extract rate-limit middleware
final fix (really)	docs(readme): document local Docker setup

```
fix(auth): refresh token before expiry check

Tokens expiring mid-request caused 401s on slow networks.
Refresh when less than 60 s remain instead of after expiry.

Refs: AUTH-311
```

Clean up before you share

$ git rebase -i origin/main

```
pick 3f2a1c feat(cart): add coupon field
fixup 9b7d20 fix typo
pick c41e8a test(cart): cover coupon rules
squash 77ab03 more tests
reword 1d09f4 wip
```

Five noisy commits become three that each make sense on their own.

git commit --fixup <sha>
then rebase -i --autosquash folds it in

git add -p
stage hunks to keep commits atomic

pull.rebase = true
no “Merge branch main” noise

push --force-with-lease
safe after a rebase; never plain --force

squash or rebase merge
keeps main linear and readable

04 · GIT AT SCALE

When the repository gets heavy

PROBLEM	TOOL	COMMAND
Large binaries: models, datasets, design files, videos	**Git LFS**	`git lfs track "*.psd" "*.onnx"`
Clone takes ages because of years of history	**Partial clone**	`git clone --filter=blob:none <url>`
You only work in one part of a monorepo	**Sparse checkout**	`git sparse-checkout set services/api`
CI only needs the latest code	**Shallow clone**	`git clone --depth 1 <url>`
Build outputs and generated files committed	**.gitignore**	`dist/ build/ *.log → artifact store`

04 · GIT AT SCALE

Monorepo or many repos?

Monorepo

GAINS

Atomic changes across services · one CI setup · shared tooling and code

COSTS

Needs sparse checkout, CODEOWNERS and path-filtered CI to stay fast

Polyrepo

GAINS

Small, fast repos · independent release cycles · clear ownership

COSTS

Cross-repo changes need coordination; shared library versions drift

Sharing code between repos

Prefer a versioned package in a registry. Use **submodules** to pin external code at an exact commit; **subtree** when consumers shouldn't need extra commands.

04 · GIT AT SCALE

Keeping Git fast

`git maintenance start`

Background prefetch, commit-graph and repacking, so fetch and log stay quick

`core.fsmonitor = true`

A file-system watcher makes git status fast on huge working trees

`feature.manyFiles = true`

Smaller index and untracked cache for repositories with many files

`scalar clone <url>`

Ships with Git: partial clone, sparse checkout and maintenance in one step

In CI: shallow fetch, cache LFS objects between runs, and trigger jobs only for the paths that changed.

05 · PROBLEMS & FIXES

Everyday accidents

PROBLEM

I committed to main instead of my branch

FIX

```
git branch feature/x
git reset --hard origin/main
git switch feature/x
```

Only if main isn't pushed yet

PROBLEM

My last commit is missing a file

FIX

```
git add missed.ts
git commit --amend --no-edit
```

Already pushed? Make a new commit instead

PROBLEM

I lost commits after a reset or rebase

FIX

```
git reflog
git branch rescue HEAD@{2}
```

Reflog keeps every move of HEAD for about 90 days

Team-level incidents

PROBLEM

An API key was pushed to the repo

FIX

```
git filter-repo --path .env
--invert-paths
git push --force-with-lease
```

Rotate the key first. Add gitleaks to pre-commit.

PROBLEM

A 3-week-old branch conflicts everywhere

FIX

```
git fetch origin
git rebase origin/main
```

Sync daily, split into smaller PRs, enable rerere.

PROBLEM

v1.5 broke something. Which commit?

FIX

```
git bisect start
git bisect bad v1.5.0
git bisect good v1.4.0
git bisect run npm test
```

Binary search finds it in a few steps.

Six habits that prevent conflicts

01

Small PRs, merged fast

A few hundred lines, reviewed within a day

02

Sync every day

Rebase on main each morning; small conflicts stay small

03

Clear ownership

CODEOWNERS and module boundaries limit overlapping edits

04

Auto-format everything

Prettier, Black or gofmt in pre-commit: no whitespace fights

05

Defuse hot files

Split giant configs; generate changelogs in CI, not by hand

06

Reuse resolutions

rerere replays a conflict fix you already made once

06 · CONFLICTS & TRACKING

From ticket to release

TICKET
`PAY-142`
The why

→

BRANCH
`feature/PAY-142`
Linked by name

→

COMMITS
`Refs: PAY-142`
Typed messages

→

PULL REQUEST
`Closes PAY-142`
Review + CI

→

RELEASE
`v1.5.0`
Tag + changelog

Protect main

Required reviews and status checks; no direct or force pushes

Tag every release

Semantic versions, so any build maps to an exact commit

Automate the changelog

release-please or semantic-release read your commit types

Small branches. Clear messages. Merge often.

A clean history isn’t a tool you install. It’s a habit the whole team keeps.

KNOWLEDGE RELAY · LEARN TODAY, BUILD TOMORROW

Questions?

Anandu Babu

Lead Engineer – Software Systems · Algosium

Try this week

`git config --global pull.rebase true`

No more accidental merge commits

Adopt Conventional Commits

Add commitlint to one repo and see how it feels

Protect main

Require a review and green CI before merge